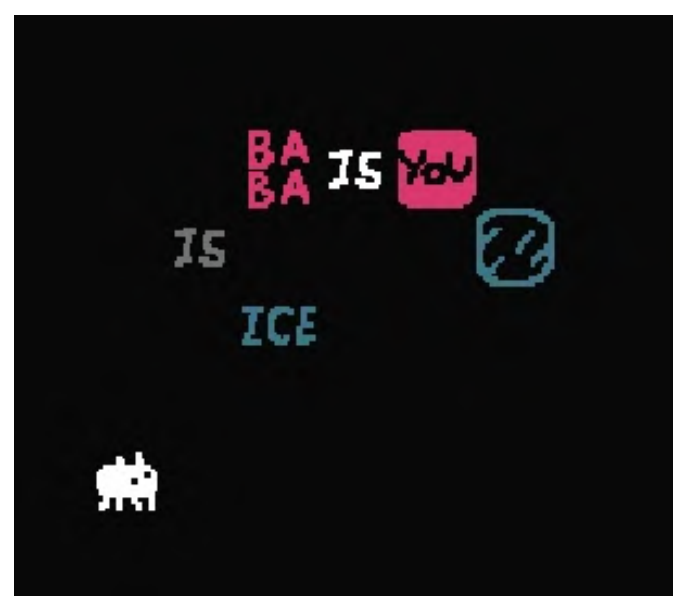


BA BA IS AL



AM
AN

ARB
AAI

LI
ZA

RA
NGA

the problem

a game where you change the rules ... ?

- rules of how the game works are part of the map
- character (baba, initially) can move around and change the rule blocks
- objective: reach a 'state' where 'you is win' is somehow true
- problems:
 - how to address changing rules using RL?
 - how to make the agent understand the rules?
- agent's actions also have the potential to form better suited rules, so rule changes can be desirable!



rl formulation

the states, actions and rewards

State Representation

- **Grid Tensor ($G_t \in \mathbb{R}^{n \times m \times k}$)**: one-hot encoded game objects per cell
- **Rule Tensor ($R_t \in \mathbb{N}^{r \times 3}$)**: active rules as (subject, verb, property) triples
- **Mask Tensor ($M_t \in \{0,1\}^r$)**: distinguishes valid from padded rules

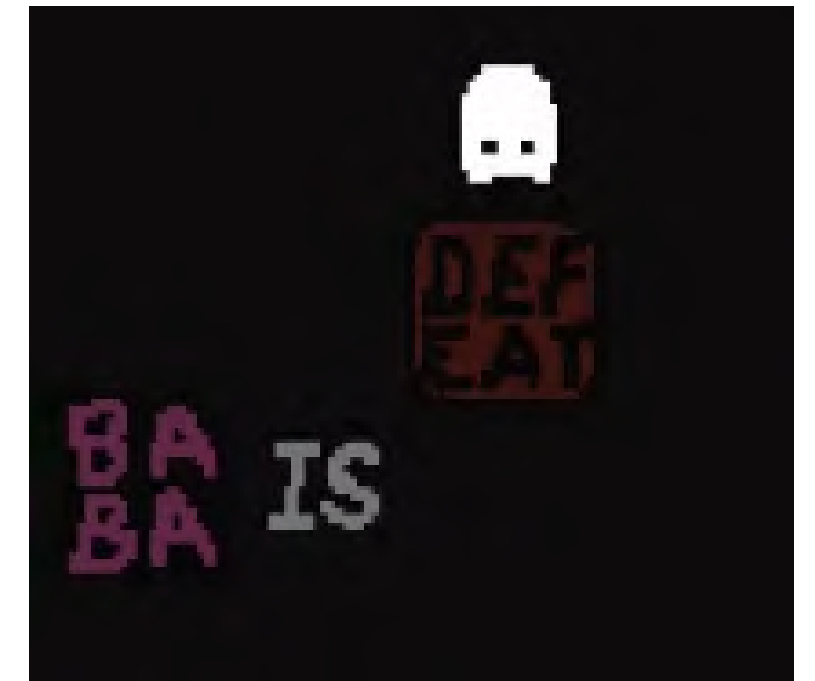
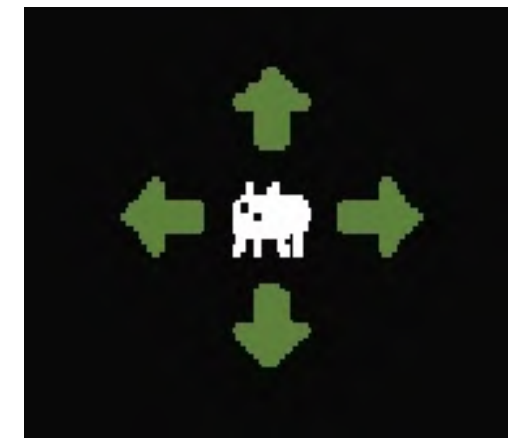
Action Space

- Discrete actions: {up, down, left, right}

Reward Function

- -0.01 per step (to encourage efficiency)
- -0.05 for redundant moves
- -1 for defeat/triggering lose
- +5 for win

Denser rewards tested but led to **reward hacking**, e.g., agents repeated meaningless rule changes – failed to generalise beyond level-specific cues.



related work & literature review

what are the current methods?

- 1. first work built a simulator and trains two algorithms – **REINFORCE & DQN**
 - trained on simple levels where rule changing is not needed to win
 - algorithm trains only to win a specific level! not a general playing agent
- 2. second work uses **proximal policy optimisation**
 - suggest adding attention layers to add text semantics when converting environment into state vector
 - no results : more examples and iterations needed! (their claim)
- 3. use **LLMs for instructions**
 - llms receive visual input and give plan of 3 kinds of high level instructions : goto(place), make(rule), break(rule)
 - is really aimed at benchmarking LLMs and not specifically solving baba is you puzzles

Model	Accuracy (mean ± std)
gemini-1.5-flash	20.0 ± 29.28
gemini-1.5-pro	14.67 ± 20.66
gpt-4o	17.33 ± 28.15

[1] <https://github.com/utilForever/baba-is-auto> (2023)
[2] https://github.com/uvulab/baba_rl_intro (2022)
[3] Baba is AI: Break the rules to beat the benchmark. July, 2024.

related work & literature review

what are the current methods?

4. this work by far ai trains an **RNN** to play Sokoban, more so for interpretability
- train a Deep RepeatingConvLSTM using the IMPALA rl algorithm
 - found that models benefit from a bunch of 'thinking' or no action steps at the start, to think ahead
 - 1.28 m parameter model for 10 x 10 sized Sokoban levels. add in our linguistic rules and we need a really really big model!

5. a very recent work on **Sokoban uses HRL**

- joint training of 6 subgoal based hierarchies in a combined 0.7 m parameter model
- adding search and sample inferencing (i.e. picking best output) enhances results

Policy	Solved %	Targets	Searches	HalfWeg solution length
<i>PL</i> ₂	0.4	1	-	14.5
<i>PL</i> ₃	3.7	1	-	21.4
<i>PL</i> ₄	8.3	1	-	37.0
<i>PL</i> ₅	12.9	1	-	65.6
<i>PL</i> ₂	0.6	36	-	14.2
<i>PL</i> ₃	10.6	36	-	25.1
<i>PL</i> ₄	31.3	36	-	50.1
<i>PL</i> ₅	46	36	-	102.8
<i>PL</i> ₅	79.4	36	100	189.4
<i>PL</i> ₅	90.2	36	1000	214.7

Table 1: Summary of trained agent performance on the Boxoban puzzle

[4] Taufeeque, M., Quirke, P., Li, M., Cundy, C., Tucker, A. D., Gleave, A., & Garriga-Alonso, A. (2024). Planning in a recurrent neural network that plays Sokoban. ArXiv.org. <https://arxiv.org/abs/2407.15421>

[5] Pastukhov, S. (2025). Solving Sokoban using Hierarchical Reinforcement Learning with Landmarks. ArXiv.org. <https://arxiv.org/abs/2504.04366>

our thoughts

Three main models tested

1. DQN (baseline)

- CNN-based Q-function approximation with experience replay and ϵ -greedy exploration.
Struggles with generalisation and sparse rewards.

2. PPO + Attention

- **Dual-stream architecture:** CNN for spatial grid + Transformer encoder for symbolic rules

Outperformed other models in curriculum generalisation

3. HRL

- **2-level agent:** Manager (sets subgoals) + Worker (executes primitive actions)

Strong generalisation potential; longer training required

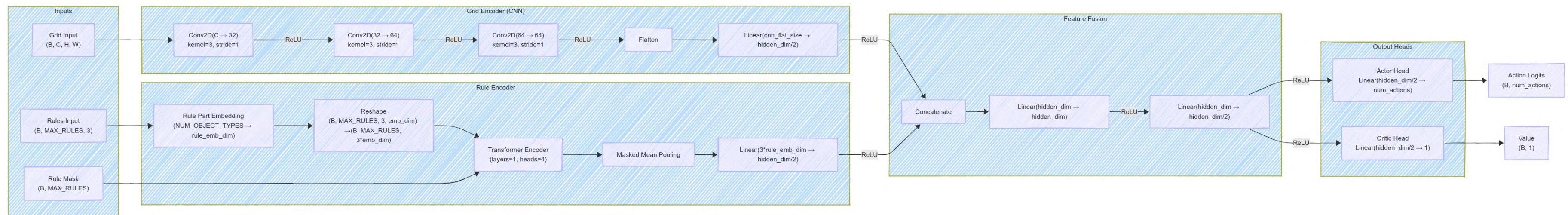
Training Strategy: Curriculum Learning

- Custom levels with increasing complexity
- Gradual introduction of objects/rules
- Enables progressive symbolic learning and generalization

what we did try....

attention + ppo

- **Idea:** Process both spatial and symbolic information effectively.
- **Architecture: Dual-Stream**
 - **Grid:** A CNN encodes spatial layout.
 - **Rules:** A Transformer encodes symbolic rules.
- **Fusion:** Combined understanding informs action & state value.



what we did try....

attention + ppo

Why Attention for Rules?

- Rules are structured sequences, not just keywords.
- Self-Attention (in Transformer) captures relationships *within* and *between* active rules.
- Naturally handles varying numbers of rules (via masking).
- Moves beyond presence/absence to *contextual understanding*

Training Strategy: PPO + Curriculum Learning

- **PPO:** Stable & efficient RL algorithm for training the Actor-Critic.
- **Curriculum: Essential!** Trained on levels of *gradually increasing complexity*.

Agent mastered simpler stages before tackling harder ones, enabling robust learning.

what we did try....

hierarchical reinforcement learning

Key Ideas

- **Employs a 2-Level Hierarchy for Long-Horizon Planning:** Decomposes complex tasks (like solving a level) into State Subgoals $\rightarrow$ Worker actions.
- **Manager Plans at Lower Frequency (K steps):** Observes the current state (u) and generates a target *state subgoal* (w) – a complete target grid representing a desired intermediate configuration (e.g., achieving a specific rule change).
- **Worker Acts Every Step to Reach Subgoal:** Observes the current state (u) and receives the Manager's target state subgoal (w). Its goal is to take primitive actions that transform state u towards the target state w .

what we did try....

hierarchical reinforcement learning

Key Ideas

- **Employs a 2-Level Hierarchy for Long-Horizon Planning:** Decomposes complex tasks (like solving a level) into State Subgoals $\rightarrow$ Worker actions.
- **Manager Plans at Lower Frequency (K steps):** Observes the current state (u) and generates a target *state subgoal* (w) – a complete target grid representing a desired intermediate configuration (e.g., achieving a specific rule change).
- **Worker Acts Every Step to Reach Subgoal:** Observes the current state (u) and receives the Manager's target state subgoal (w). Its goal is to take primitive actions that transform state u towards the target state w .
- **State Encoding (Shared):** Similar to PPO, uses a CNN to encode the spatial grid information and a Transformer to encode the active rules for any given state (u or w).
- **Learning Signals:**
 - The Worker is trained via PPO to reach the subgoal w while maximising environment rewards.
 - The Manager's value function (predicting long-term reward from state u) is trained via PPO, and its gradients implicitly train the subgoal generator network to produce useful subgoals.

what we did try....

curriculum learning

Key Ideas

Models generalized poorly with in-built game levels

- Steep learning curve
- Rapid addition of new rules and objects

Created new training and testing levels

- Gradual increase in difficulty
- New level introduces a new rule or a new object
- Each level is padded to be the same size as the largest one

curriculum learning

game levels

level 0, 1, 3, 5



MOVE

BA
BA IS YOU

FL
AG IS WIN



WA
LL IS STOP

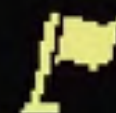
RO
CK IS PUSH



FLAG IS WIN
BABA IS YOU

ROCK IS PUSH

SKULL
IS
DEFEAT





curriculum learning

our levels

BA IS YOU



FLAG IS WIN

BA IS YOU

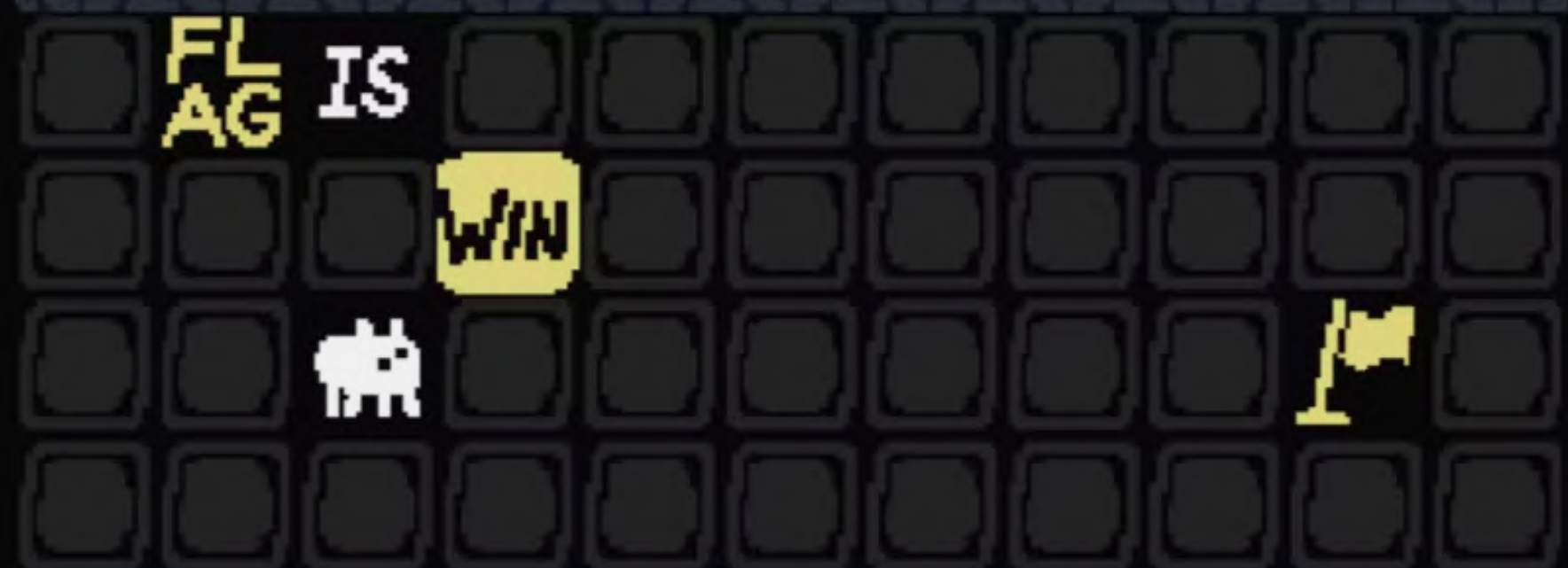
WA IS STOP



FLAG IS WIN

BA IS YOU

WA IS STOP



WA LL IS YOU

BA BA IS STOP



BA IS YOU

WA IS STOP



ROCK IS PUSH

FLAG IS WIN

BA IS YOU

WALL IS STOP



ROCK IS

FLAG IS WIN

PUSH

onwards...

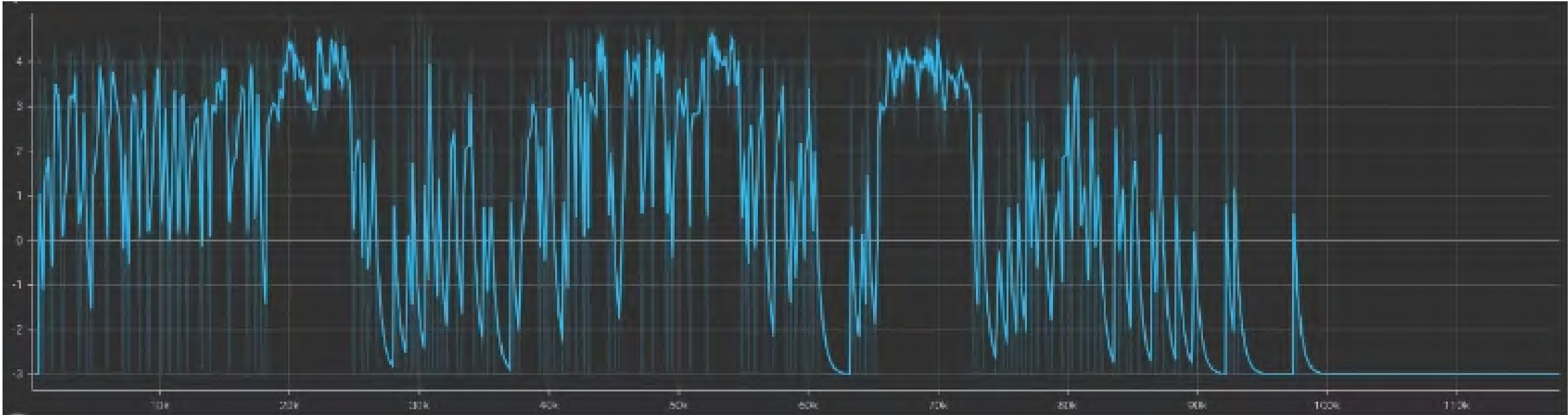
results

DQN, attention + PPO, HRL

onwards...

results

DQN



Y-Axis: Reward, X-Axis: Global Steps

onwards...

results

PPO



Example Level



Y-Axis: Rewards, X-Axis: Steps

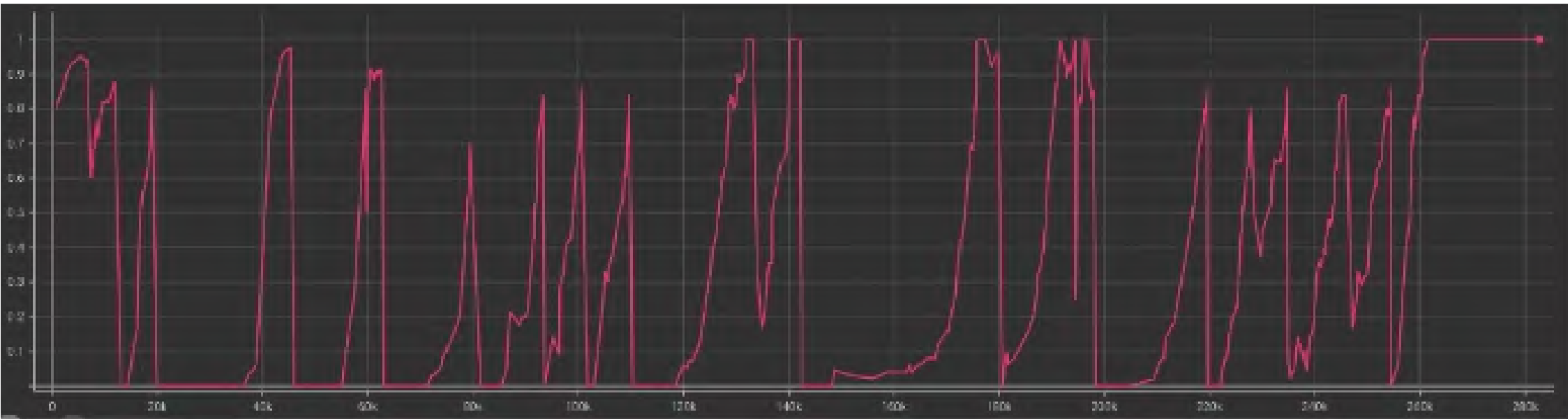
onwards...

results

PPO



Y-Axis: Episode Rewards, X-Axis: Steps

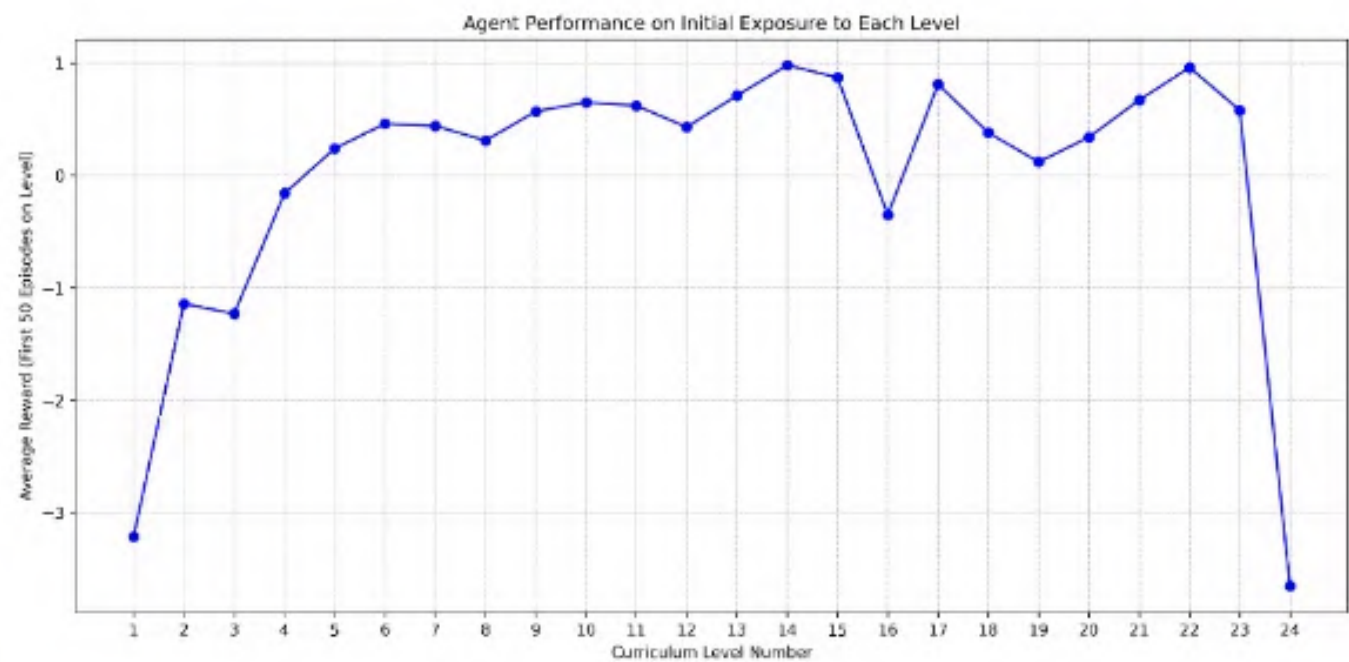


Win Rate (Window Size: 50)

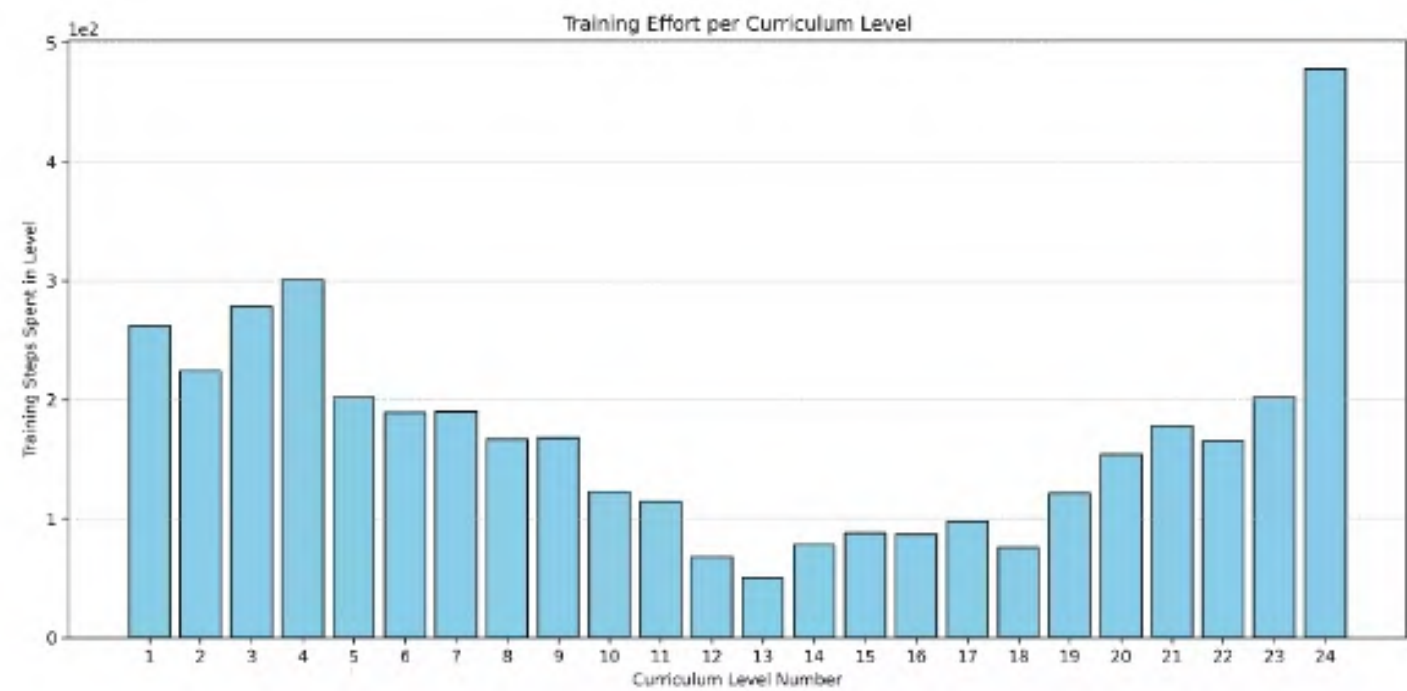
onwards...

results

HRL



Y-Axis: Avg. Reward over first 50 eps, X-Axis: Level



Y-Axis: Training steps to advance, X-Axis: Level

onwards...

results

demo

how we added

our contributions and results

Curriculum Learning

- 28 progressively complex levels
- Systematically introduced rules and objects one at a time
- Enabled generalization across harder levels

Architecture Innovations

- PPO with attention mechanism performed better than DQN
 - DQN could not solve all the curriculum levels
- Explored hierarchical reinforcement learning for abstract reasoning
- Explored adding thinking steps in PPO

lessons learnt

baba is you should be an agi benchmark

1. curriculum learning is crucial
2. incremental rule and environment changes and ensuring these are in an explorable vicinity are key
3. reward shaping is very tricky
4. need to always consider how model can hack the system and plan accordingly
5. specialised architectures for specialised problems
6. deep learning + rl can be really powerful

BAMI
BAGHT BE AGI

BA IS THANKS

references

our dupes

- [1] Baba is AI: Break the rules to beat the benchmark. (n.d.). <https://arxiv.org/html/2407.13729v1>
- [2] M. Charity and J. Togelius, "Keke AI Competition: Solving puzzle levels in a dynamically changing mechanic space," 2022 IEEE Conference on Games (CoG), Beijing, China, 2022, pp. 570–575, doi: 10.1109/CoG51982.2022.9893650.
- [3] https://github.com/uvulab/baba_rl_intro
- [4] <https://github.com/utilForever/baba-is-auto>
- [5] <https://github.com/MasterMilkX/KekeCompetition>
- [6] Madureira, B., & Schlangen, D. (2020). An overview of natural language state representation for reinforcement learning. arXiv preprint arXiv:2007.09774.
- [7] Taufeeque, M., Quirke, P., Li, M., Cundy, C., Tucker, A. D., Gleave, A., & Garriga-Alonso, A. (2024). Planning in a recurrent neural network that plays Sokoban. ArXiv.org. <https://arxiv.org/abs/2407.15421>
- [8] Pastukhov, S. (2025). Solving Sokoban using Hierarchical Reinforcement Learning with Landmarks. ArXiv.org. <https://arxiv.org/abs/2504.04366>



Want to make a presentation like this one?

Start with a fully customizable template, create a beautiful deck in minutes, then easily share it with anyone.

Create a presentation (It's free)